

Quantium analysis

Uchechukwu Okwudili

2026-07-11

1. Introduction

This report analyses transactions and consumer behavior of retail customers who are buying chips in Quantium across a 12-month period from July 2018 to June 2019.

Loading packages

```
library(tidyverse)
library(here)
library(janitor)
library(skimr)
library(readxl)
library(stringr)
```

Reading in the data

```
customerData <- read_csv(here("data", "QVI_purchase_behaviour.csv"))
transactionData <- read_excel(here("data", "QVI_transaction_data.xlsx"))
```

Exploring the data

```
glimpse(customerData)
```

```
## Rows: 72,637
## Columns: 3
## $ LYLTY_CARD_NBR    <dbl> 1000, 1002, 1003, 1004, 1005, 1007, 1009, 1010, 1011, ~
## $ LIFESTAGE         <chr> "YOUNG SINGLES/COUPLES", "YOUNG SINGLES/COUPLES", "YO~
## $ PREMIUM_CUSTOMER <chr> "Premium", "Mainstream", "Budget", "Mainstream", "Mai~
```

```
head(customerData)
```

```
## # A tibble: 6 x 3
##   LYLTY_CARD_NBR LIFESTAGE          PREMIUM_CUSTOMER
##   <dbl> <chr>          <chr>
## 1      1000 YOUNG SINGLES/COUPLES Premium
## 2      1002 YOUNG SINGLES/COUPLES Mainstream
## 3      1003 YOUNG FAMILIES      Budget
## 4      1004 OLDER SINGLES/COUPLES Mainstream
## 5      1005 MIDAGE SINGLES/COUPLES Mainstream
## 6      1007 YOUNG SINGLES/COUPLES Budget
```

```
glimpse(transactionData)
```

```
## Rows: 264,836
## Columns: 8
## $ DATE          <dbl> 43390, 43599, 43605, 43329, 43330, 43604, 43601, 43601,~
## $ STORE_NBR     <dbl> 1, 1, 1, 2, 2, 4, 4, 4, 5, 7, 7, 8, 9, 13, 19, 20, 20, ~
## $ LYLTY_CARD_NBR <dbl> 1000, 1307, 1343, 2373, 2426, 4074, 4149, 4196, 5026, 7~
## $ TXN_ID        <dbl> 1, 348, 383, 974, 1038, 2982, 3333, 3539, 4525, 6900, 7~
## $ PROD_NBR      <dbl> 5, 66, 61, 69, 108, 57, 16, 24, 42, 52, 16, 114, 15, 92~
## $ PROD_NAME     <chr> "Natural Chip          Compny SeaSalt175g", "CCs Nacho Ch~
## $ PROD_QTY      <dbl> 2, 3, 2, 5, 3, 1, 1, 1, 1, 2, 1, 5, 2, 1, 1, 1, 4, 1, 1~
## $ TOT_SALES     <dbl> 6.0, 6.3, 2.9, 15.0, 13.8, 5.1, 5.7, 3.6, 3.9, 7.2, 5.7~
```

```
head(transactionData)
```

```
## # A tibble: 6 x 8
##   DATE STORE_NBR LYLTY_CARD_NBR TXN_ID PROD_NBR PROD_NAME      PROD_QTY TOT_SALES
##   <dbl>   <dbl>         <dbl> <dbl>   <dbl> <chr>          <dbl>   <dbl>
## 1 43390     1         1000     1      5 Natural Chi~      2      6
## 2 43599     1         1307    348     66 CCs Nacho C~      3     6.3
## 3 43605     1         1343    383     61 Smiths Crin~      2     2.9
## 4 43329     2         2373    974     69 Smiths Chip~      5     15
## 5 43330     2         2426   1038    108 Kettle Tort~      3    13.8
## 6 43604     4         4074   2982     57 Old El Paso~      1     5.1
```

```
skim(transactionData)
```

Table 1: Data summary

Name	transactionData
Number of rows	264836
Number of columns	8
Column type frequency:	
character	1
numeric	7
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
PROD_NAME	0	1	17	40	0	114	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
DATE	0	1	43464.04	105.39	43282.0	43373.0	43464.0	43555.0	43646	
STORE_NBR	0	1	135.08	76.78	1.0	70.0	130.0	203.0	272	
LYLTY_CARD_NBR	1	135549.48	80579.98	1000.0	70021.0	130357.5	203094.2	2373711		
TXN_ID	0	1	135158.31	78133.03	1.0	67601.5	135137.5	202701.2	2415841	
PROD_NBR	0	1	56.58	32.83	1.0	28.0	56.0	85.0	114	
PROD_QTY	0	1	1.91	0.64	1.0	2.0	2.0	2.0	200	
TOT_SALES	0	1	7.30	3.08	1.5	5.4	7.4	9.2	650	

```
n_distinct(transactionData$PROD_NAME)
```

```
## [1] 114
```

These functions show a brief insight into what kind of data i am dealing with. the number of rows and columns, the formatting for each column, the headings, and so on.

2. Data cleaning

2.1 Transaction Data

2.1.1 Standardizing transaction data

At first glance, I saw that the 'DATE' column was not properly formatted. I had to correct that. Also, the number of unique product names was too long, so I narrowed it down in the next few steps from 113 to 27 by splitting the product name column to extract the brand names, and removing products that are not chips.

```
cleanTransaction <- transactionData %>%
  filter(!grepl("dip",PROD_NAME, ignore.case = TRUE)) %>%
  filter(!grepl("cheese",PROD_NAME, ignore.case = TRUE)) %>%
  filter(!(PROD_NAME %in% c("Woolworths Mild Salsa 300g", "Woolworths Medium Salsa 300g"))) %>%
  mutate DATE = as.Date(DATE, origin = "1899-12-30") %>%
  clean_names() %>%
  separate(prod_name, into = c("Brand", "Description"), sep = " ", extra = "merge") %>%
  mutate(pack_size = str_extract(Description, "[0-9]+g"))
```

Using the filter function, I removed the products that are not chips, but rather dips, sauces and cheeses. By using the mutate function, I formatted the date correctly, by instructing R to replace the "DATE" column with a new calculated version (using the same starting point Excel CSV uses), and to convert this new one into an actual date format. I also split the product names column into brand and description categories for easier handling of the data. Using the mutate function, I extracted the pack sizes of the products and put them in a separate column.

```
n_distinct(cleanTransaction$Brand)
```

```
## [1] 27
```

```
cleanTransaction %>%  
  count(Brand, sort = TRUE) %>%  
  print(n = 27)
```

```
## # A tibble: 27 x 2  
##   Brand      n  
##   <chr>    <int>  
## 1 Kettle   41288  
## 2 Smiths   27422  
## 3 Pringles 25102  
## 4 Doritos  15533  
## 5 Thins    14075  
## 6 RRD      11894  
## 7 Infuzions 11057  
## 8 Cobs      9693  
## 9 Tostitos  9471  
## 10 WW       8811  
## 11 Tyrrells 6442  
## 12 Grain    6272  
## 13 Natural  6050  
## 14 Red      5885  
## 15 Dorito   3185  
## 16 Twisties 3170  
## 17 Infzns   3144  
## 18 Cheetos  2927  
## 19 Woolworths 2921  
## 20 Snbts    1576  
## 21 Burger   1564  
## 22 CCs      1514  
## 23 GrnWves   1468  
## 24 Smith    1451  
## 25 Sunbites 1432  
## 26 NCC      1419  
## 27 French   1418
```

This shows that the brand names are unique and the number of transactions they are involved in. I noticed however, that the Brand “Doritos” appears twice, improperly entered as “Dorito”. And it is not the only one. “Infuzions” and “Infzns”, “Snbts” and “Sunbites”, “RRD” and “Red” appear to mean the same things.

2.1.2 Standardizing the brand name

To be sure that the two mean the same thing, I do a quick check to compare their product descriptions side by side for any similarities.

```
cleanTransaction %>%  
  filter(grepl("^Dorito", Brand, ignore.case = TRUE)) %>%  
  group_by(Description, tot_sales, prod_qty) %>%  
  distinct(Brand)
```

```
## # A tibble: 38 x 4
## # Groups:   Description, tot_sales, prod_qty [38]
##   Description          tot_sales prod_qty Brand
##   <chr>                <dbl>    <dbl> <chr>
## 1 Corn Chip Mexican Jalapeno 150g      3.9        1 Doritos
## 2 Corn Chip Southern Chicken 150g      3.9        1 Doritos
## 3 Mexicana      170g      4.4        1 Doritos
## 4 Corn Chp      Supreme 380g      3.25        1 Dorito
## 5 Corn Chip Mexican Jalapeno 150g      7.8        2 Doritos
## 6 Salsa          Medium 300g      2.6        1 Doritos
## 7 Corn Chp      Supreme 380g      6.5        2 Dorito
## 8 Corn Chips   Original 170g      4.4        1 Doritos
## 9 Corn Chip Southern Chicken 150g      7.8        2 Doritos
## 10 Corn Chips   Original 170g     13.2        3 Doritos
## # i 28 more rows
```

After confirming that they mean the same thing, I went ahead with the standardization to replace the brand names. Also, I found an outlier here. There was a transaction made with an amount of 650 to purchase 200 packets of dorito chips. This will be addressed right after the next few steps.

```
cleanTransaction <- cleanTransaction %>%
  mutate(Brand = str_replace(Brand, "^Dorito$", "Doritos"))

cleanTransaction <- cleanTransaction %>%
  mutate(Brand = str_replace(Brand, "^Infzns$", "Infuzions"))

cleanTransaction <- cleanTransaction %>%
  mutate(Brand = str_replace(Brand, "^Snbts$", "Sunbites"))

cleanTransaction <- cleanTransaction %>%
  mutate(Brand = str_replace(Brand, "^Red$", "RRD"))
```

```
cleanTransaction %>%
  filter(grepl("^Dorito", Brand, ignore.case = TRUE)) %>%
  distinct(Brand)
```

```
## # A tibble: 1 x 1
##   Brand
##   <chr>
## 1 Doritos
```

After replacing the brand name, I checked again for each brand to confirm that the code worked.

2.2 Data cleaning: Customer Data

I go ahead to standardize the column names in the customer data as well, to ensure that the case styling is consistent for both data, allowing for a smooth workflow.

```
cleanCustomer <- customerData %>%
  clean_names()
```

2.3 Merging the two Data—

The given data is split across two documents that each tell half the story. The transaction data tells us what was bought, how much it cost, when the purchase happened, and at which store. The customer data tells us who did the buying that happened as shown in the transaction data, their life stage and spending segment. By merging the data, we label each transaction with the person who made it.

```
mergedData <- cleanTransaction %>%
  left_join(cleanCustomer, by = "lylty_card_nbr")

cleanTransaction %>%
  filter(!(lylty_card_nbr %in% cleanCustomer$lylty_card_nbr))

## # A tibble: 0 x 10
## # i 10 variables: date <date>, store_nbr <dbl>, lylty_card_nbr <dbl>,
## #   txn_id <dbl>, prod_nbr <dbl>, Brand <chr>, Description <chr>,
## #   prod_qty <dbl>, tot_sales <dbl>, pack_size <chr>

sum(is.na(mergedData$premium_customer))

## [1] 0

dim(mergedData)

## [1] 226184      12

head(mergedData)

## # A tibble: 6 x 12
##   date      store_nbr lylty_card_nbr txn_id prod_nbr Brand Description prod_qty
##   <date>      <dbl>      <dbl>   <dbl>   <dbl> <chr>   <chr>      <dbl>
## 1 2018-10-17         1         1000     1       5 Natu~ Chip      ~         2
## 2 2019-05-20         1        1343    383      61 Smit~ Crinkle Cu~         2
## 3 2018-08-17         2        2373    974      69 Smit~ Chip Thinl~         5
## 4 2018-08-18         2        2426   1038     108 Kett~ Tortilla C~         3
## 5 2019-05-16         4        4149   3333      16 Smit~ Crinkle Ch~         1
## 6 2019-05-16         4        4196   3539      24 Grain Waves      ~         1
## # i 4 more variables: tot_sales <dbl>, pack_size <chr>, lifestage <chr>,
## #   premium_customer <chr>
```

After merging, I checked the data to ensure that it matched correctly. I used the filter function to show every transaction where the loyalty card number does NOT appear in my customer list, which returned 0. I double check with the summary function to show how many transactions ended up with no matching customer information after the merge (Count how many rows have a missing value in the premium_customer column), which also returned 0. So we are good to go!

2.3.1 Removing outliers

Recall the outlier that we found from the previous standardization process.

```
mergedData %>%
  filter(Brand == "Doritos" & tot_sales > 100)
```

```
## # A tibble: 2 x 12
##   date      store_nbr lyqty_card_nbr txn_id prod_nbr Brand Description prod_qty
##   <date>      <dbl>      <dbl> <dbl>   <dbl> <chr> <chr>      <dbl>
## 1 2018-08-19      226      226000 226201     4 Doritos Corn Chp ~      200
## 2 2019-05-20      226      226000 226210     4 Doritos Corn Chp ~      200
## # i 4 more variables: tot_sales <dbl>, pack_size <chr>, lifestage <chr>,
## #   premium_customer <chr>
```

It seems there are only two occasions where those bulk purchases were made, from the same customer too. It was likely a commercial buy for their own retail stores, however, if we allow it, these two data entry will skew our analysis, so I went ahead to remove them. But first, i need to make sure it's not just the "Doritos" that our biggest customer here has purchased.

```
mergedData %>%
  filter(lylty_card_nbr == 226000)
```

```
## # A tibble: 2 x 12
##   date      store_nbr lyqty_card_nbr txn_id prod_nbr Brand Description prod_qty
##   <date>      <dbl>      <dbl> <dbl>   <dbl> <chr> <chr>      <dbl>
## 1 2018-08-19      226      226000 226201     4 Doritos Corn Chp ~      200
## 2 2019-05-20      226      226000 226210     4 Doritos Corn Chp ~      200
## # i 4 more variables: tot_sales <dbl>, pack_size <chr>, lifestage <chr>,
## #   premium_customer <chr>
```

Those two seem to be the only ones, so I filtered out this customer's Loyalty card number to avoid misrepresentation in my further analysis.

```
mergedData <- mergedData %>%
  filter(lylty_card_nbr != 226000)
```

Then I used the summary function to check for any other outliers, and inconsistencies

```
summary(mergedData)
```

```
##           date           store_nbr      lylty_card_nbr      txn_id
##  Min.   :2018-07-01   Min.    : 1.0   Min.    : 1000   Min.    : 1
## 1st Qu.:2018-09-30   1st Qu.: 70.0   1st Qu.: 70044   1st Qu.: 67748
## Median :2018-12-30   Median :130.0   Median : 130345   Median : 135090
## Mean   :2018-12-29   Mean   :135.1   Mean    : 135590   Mean    : 135185
## 3rd Qu.:2019-03-31   3rd Qu.:203.0   3rd Qu.: 203086   3rd Qu.: 202662
## Max.   :2019-06-30   Max.    :272.0   Max.    :2373711   Max.    :2415841
##      prod_nbr      Brand      Description      prod_qty
##  Min.    : 1.00   Length:226182   Length:226182   Min.    :1.000
## 1st Qu.: 29.00   Class :character   Class :character   1st Qu.:2.000
## Median : 52.00   Mode  :character   Mode  :character   Median :2.000
## Mean    : 56.94                                     Mean    :1.906
## 3rd Qu.: 87.00                                     3rd Qu.:2.000
## Max.    :114.00                                     Max.    :5.000
```

```
##      tot_sales      pack_size      lifestage      premium_customer
## Min.       : 1.500   Length:226182   Length:226182   Length:226182
## 1st Qu.: 5.700   Class :character   Class :character   Class :character
## Median : 7.400   Mode  :character   Mode  :character   Mode  :character
## Mean      : 7.182
## 3rd Qu.: 8.800
## Max.      :29.500
```

There appears to be no other derangement in the data.

3. Analysis and Findings

Loading packages for visualizations

```
library(ggbeeswarm)
library(RColorBrewer)
library(ggtext)
library(magick)
library(beepr)
library(ggplot2)
```

Setting the plot theme

```
theme_set(theme_bw())
```

3.1 Transaction trend over time

What does the customer spending look like over time?. Are customers spending more and more each month with an upward trend? Is there an expected seasonal spike?

```
mergedData %>%
  count(date)
```

```
## # A tibble: 364 x 2
##   date      n
##   <date>   <int>
## 1 2018-07-01 601
## 2 2018-07-02 602
## 3 2018-07-03 609
## 4 2018-07-04 621
## 5 2018-07-05 593
## 6 2018-07-06 659
## 7 2018-07-07 635
## 8 2018-07-08 605
## 9 2018-07-09 641
## 10 2018-07-10 587
## # i 354 more rows
```

This shows a summary of the number of transactions occurring on each date. There was no less than 500 purchases made on each given day. However, instead of a count of 365, which is the number of days in a year, it is giving only 364. I went ahead to identify the missing date and include it.


```

allDates <- data.frame(
  DATE = seq(as.Date("2018-07-01"), as.Date("2019-06-30"), by = "day")) %>%
  clean_names()

transactionsByDate <- mergedData %>%
  count(date)

transactionsByDate <- allDates %>%
  left_join(transactionsByDate, by = "date")

transactionsByDate <- transactionsByDate %>%
  mutate(n = ifelse(is.na(n), 0, n))

transactionsByDate %>%
  filter(n == 0)

##           date n
## 1 2018-12-25  0

```

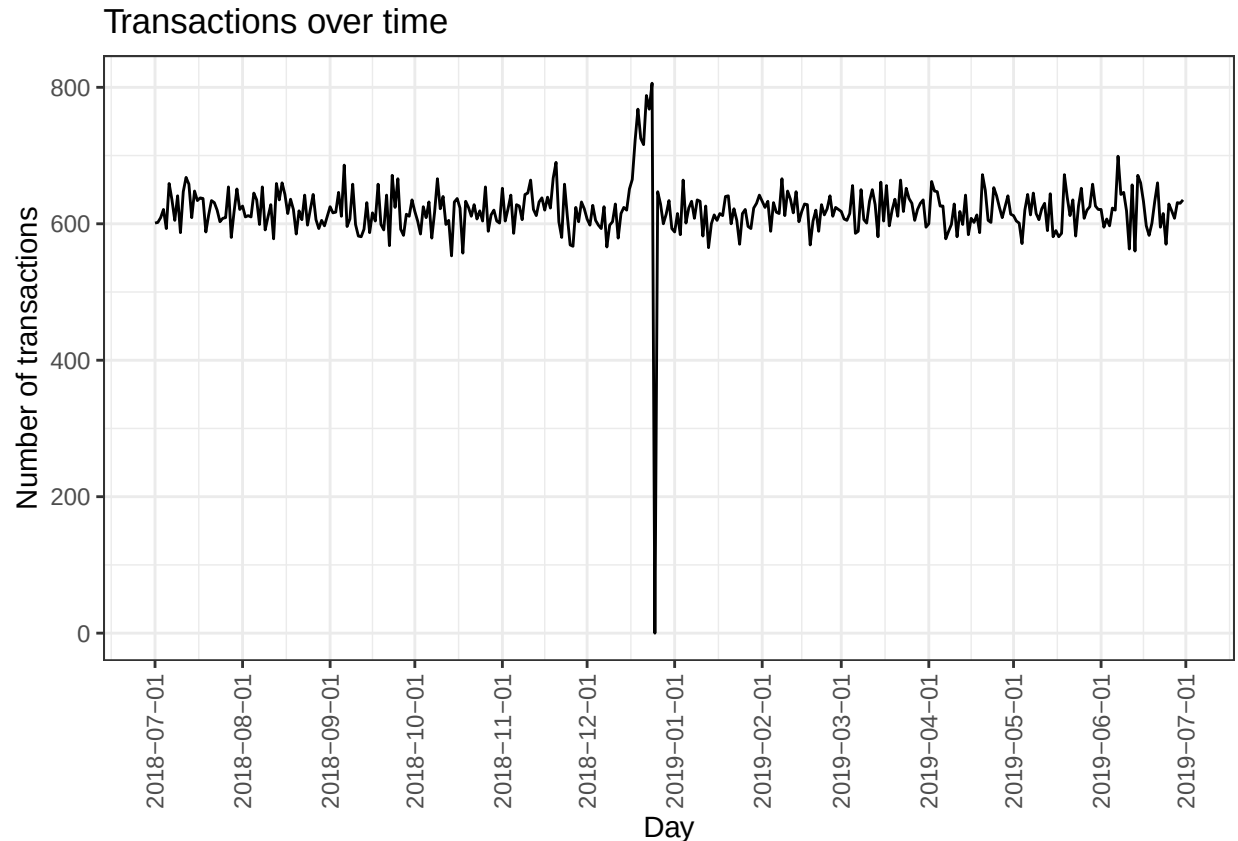
I created a sequence of dates from 1 Jul 2018 to 30 Jun 2019 and then joined it with the count of transactions by date, extracted into a different column. For our missing data to not return as null, I used the mutate function to put the transaction count at 0. After viewing with the filter function, it appears that the missing date is less likely an error. The date shown to have zero transactions is the 25th of December, 2018 (Christmas), and we assume that on this day, the stores were closed, which is why no sales were recorded.

3.1.1 Plot transactions over time

```

ggplot(transactionsByDate, aes(x = date, y = n)) +
  geom_line() +
  labs(x = "Day", y = "Number of transactions", title = "Transactions over time") +
  scale_x_date(breaks = "1 month") +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5))

```



The volume of transactions stayed relatively consistent throughout the July 2018 to June 2019 period, averaging approximately 660 transactions in each month. A significant drop was observed in December 2018, which is seen as a zero transaction count in late December, corresponding with our initial findings of zero sales made on the 25th of that month. However, a significant seasonal spike was observed in early and mid December 2018, with about 33% increase in sales, implying an increased consumer spending during the festive period. *I would recommend that the chips manager prioritize stock availability and promotional activity in November to capitalize on this increased seasonal demand.*

3.2 December transactions

Now I am taking a closer look at December, which is the month with the most amount of change in transaction pattern, to see if it has more information for us.

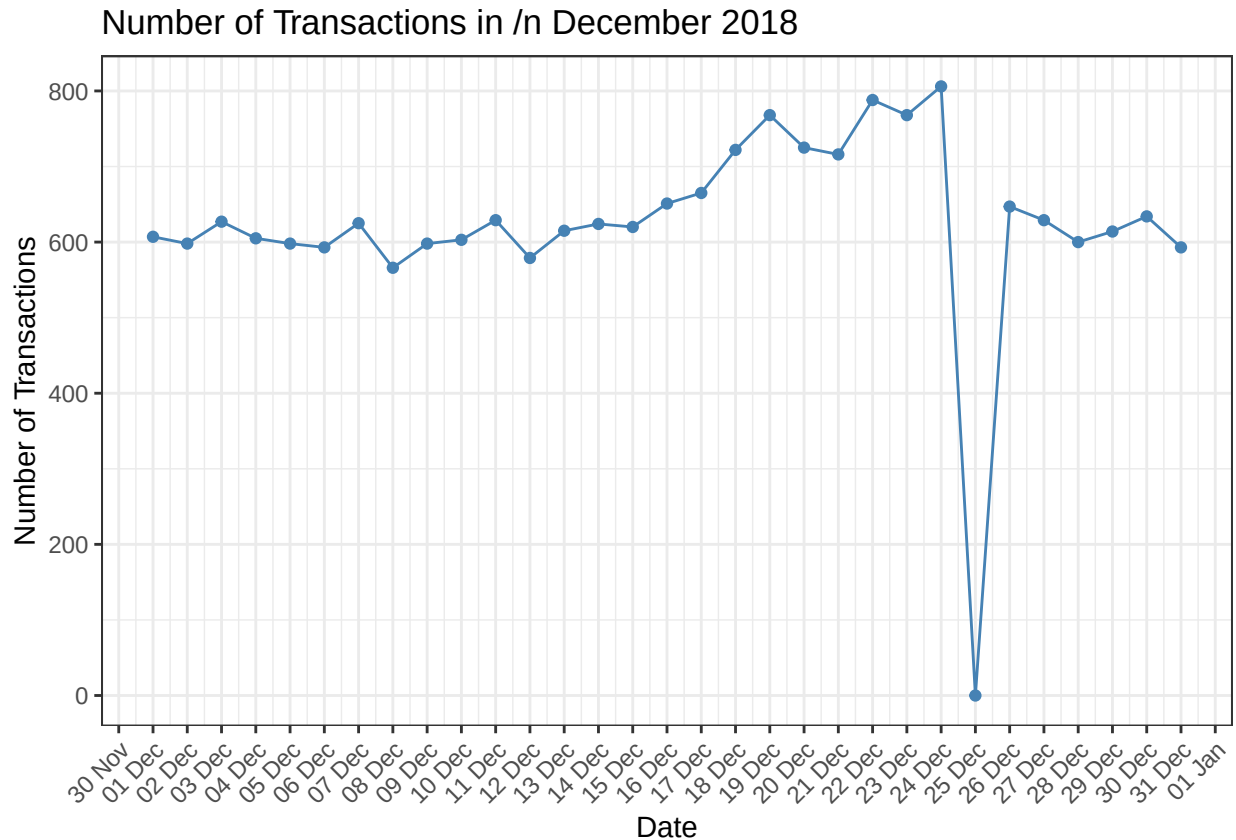
```
decemberData <- transactionsByDate %>%
  filter(date >= as.Date("2018-12-01") & date <= as.Date("2018-12-31"))
```

Here, I extract December from the full year's data, and keep only the rows where the date falls within December 2018, removing everything before 1st December and after 31st December.

3.2.1 Plot December transaction data

```
ggplot(decemberData, aes(x = date, y = n)) +
  geom_line(color = "steelblue") +
```

```
geom_point(color = "steelblue") +
labs(title = "Number of Transactions in /n December 2018",
     x = "Date",
     y = "Number of Transactions") +
scale_x_date(date_breaks = "1 day", date_labels = "%d %b") +
theme_bw() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The sales volume start to rise above the baseline from the 15th of December, and increases each day, with the highest volume seen on the 24th of December, right before the drop to zero on the day no sales were made because it was a holiday. By the 26th, transactions continued, and rapidly regained momentum, but doesn't go above the baseline area again, suggesting that the surge was a temporary one due to the festive period. *The pre-Christmas period is a significant opportunity to maximize sales. I would recommend that promotional activity and stock planning should be enhanced in this period, starting from the first week of December, to fully capitalize on the festive demand. There may also be a need for targeted promotions in January to sustain momentum.*

3.3 Consumer behaviour

Who makes up Quantum's biggest consumer base? Which customer segment drives the most growth? Are the customers family size related to the amount of purchase they make? What products do the customers prefer?

3.3.1 Transaction count by spending habit of customers

```
mergedData %>%  
  count(premium_customer, sort = TRUE)
```

```
## # A tibble: 3 x 2  
##   premium_customer     n  
##   <chr>             <int>  
## 1 Mainstream       87026  
## 2 Budget           79471  
## 3 Premium          59685
```

```
mergedData %>%  
  filter(premium_customer == "Mainstream") %>%  
  count(lifestage, sort = TRUE)
```

```
## # A tibble: 7 x 2  
##   lifestage             n  
##   <chr>             <int>  
## 1 RETIREES          18310  
## 2 YOUNG SINGLES/COUPLES 17716  
## 3 OLDER SINGLES/COUPLES 15665  
## 4 OLDER FAMILIES     12212  
## 5 YOUNG FAMILIES     11052  
## 6 MIDGE SINGLES/COUPLES 10087  
## 7 NEW FAMILIES        1984
```

I summarize the customer data to show the relationship between the customer's life stage and their purchasing habits. Most customers are mainstream buyers, while the Premium buyers are the least, with a difference of about 27341 transactions compared to each other. As for their life stages, the Retirees seem to have a bigger affinity for chips than all the others, and New families are lagging by tens of thousands in chips purchase compared to other customers. I encountered a problem however. There appears to be a row that does not contain information on a customer transaction.

3.3.2 Removing null rows

```
mergedData %>%  
  filter(is.na(Brand)) %>%  
  select(Brand, Description) %>%  
  distinct()
```

```
## # A tibble: 0 x 2  
## # i 2 variables: Brand <chr>, Description <chr>
```

I checked to see if the missing data is just from the consumer data, perhaps something I missed after merging both the transaction and customer data. But it seems like both were affected, so I have to filter it out.

```
mergedData <- mergedData %>%
  filter(!is.na(Brand))
```

After doing this, I went back to re-run the premium customer line of code to see if it worked, and it did.

3.3.3 Summary of product transactions

```
mergedData %>%
  group_by(Brand) %>%
  summarise(minsales = min(tot_sales, na.rm = TRUE),
            maxsales = max(tot_sales, na.rm = TRUE),
            meansales = mean(tot_sales, na.rm = TRUE)) %>%
  arrange(-meansales)
```

```
## # A tibble: 23 x 4
##   Brand      minsales maxsales meansales
##   <chr>      <dbl>    <dbl>    <dbl>
## 1 Kettle      4.2      27      9.45
## 2 Twisties    4.6      23      8.79
## 3 Tostitos    4.4      22      8.42
## 4 Doritos     2.6      22      8.20
## 5 Tyrrells    4.2      21      8.02
## 6 Smiths      2.6     29.5      7.40
## 7 Cobs        3.8      19      7.28
## 8 Pringles    3.7     18.5      7.08
## 9 Infuzions   2.4      19      6.97
## 10 Grain      3.6      18      6.86
## # i 13 more rows
```

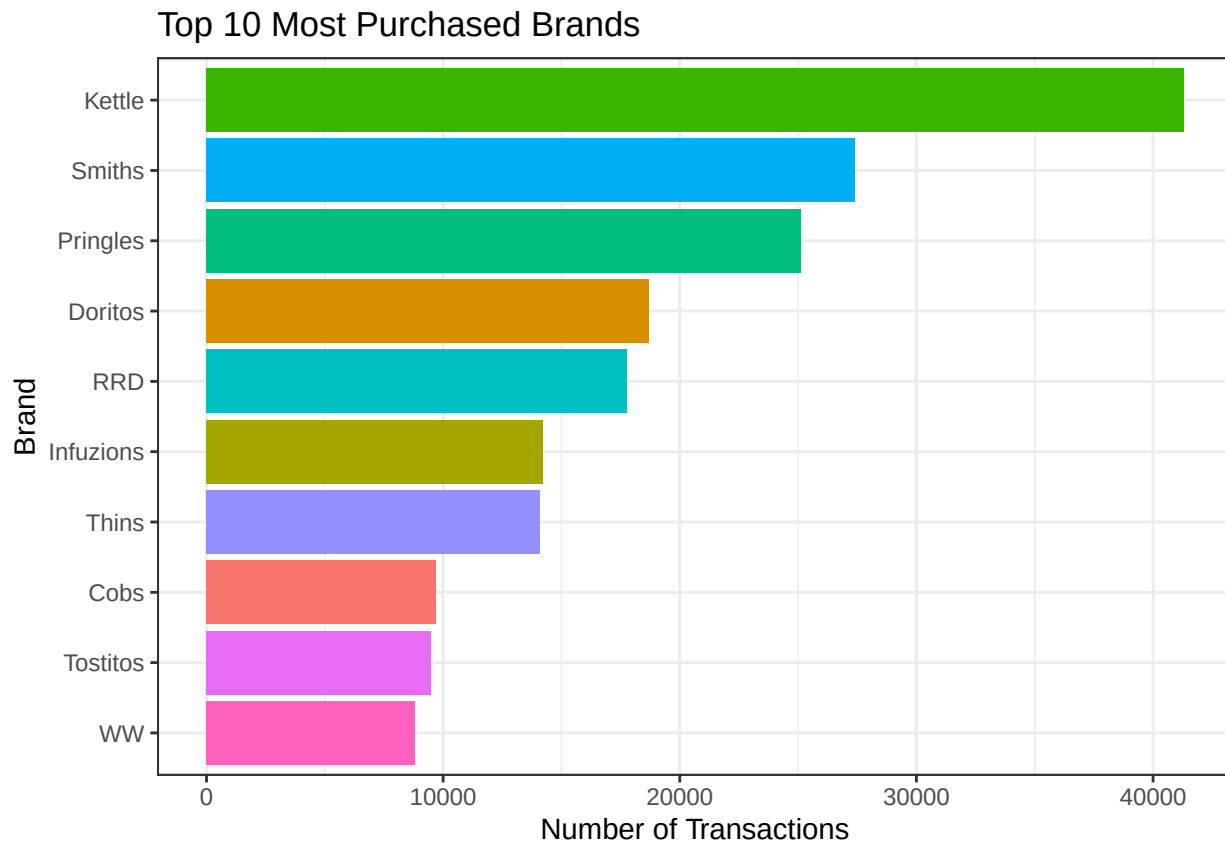
I tried to show the minimum, maximum, and average sales count for each of the unique product brands. It revealed a variation in price consistency for many products, but this could have been affected by discount rates, the quantity of products bought, or varying pack sizes and numbers. Also, the mean sales help to tell us which products might be premium, and which might be better on the budget, with “Woolworths” having the least revenue, and “Kettle” sitting at number one.

3.3.4 What kind of chips do customers prefer?

We need to know the our business’s strengths and which products are driving sales the most.

```
mergedData %>%
  group_by(Brand) %>%
  summarise(
    total_transactions = n(),
    total_revenue = sum(tot_sales, na.rm = TRUE)
  ) %>%
  arrange(desc(total_transactions)) %>%
  head(10) %>%
  ggplot(aes(x = reorder(Brand, total_transactions),
              y = total_transactions,
              fill = Brand)) +
```

```
geom_bar(stat = "identity") +
coord_flip() +
labs(title = "Top 10 Most Purchased Brands",
      x = "Brand",
      y = "Number of Transactions") +
theme_bw() +
theme(legend.position = "none")
```



It appears that Kettles, Smiths, Pringles, Doritos, and Thins are the most loved chips brands, with Kettle leading the statistics by a transaction count of over 10000. *I would recommend that these brands take priority when stocking chips, making available some extra amounts, because they are more likely to get sold out than many of the other brands, and they will help increase sales as well.*

3.3.5 What customer segments should be prioritized

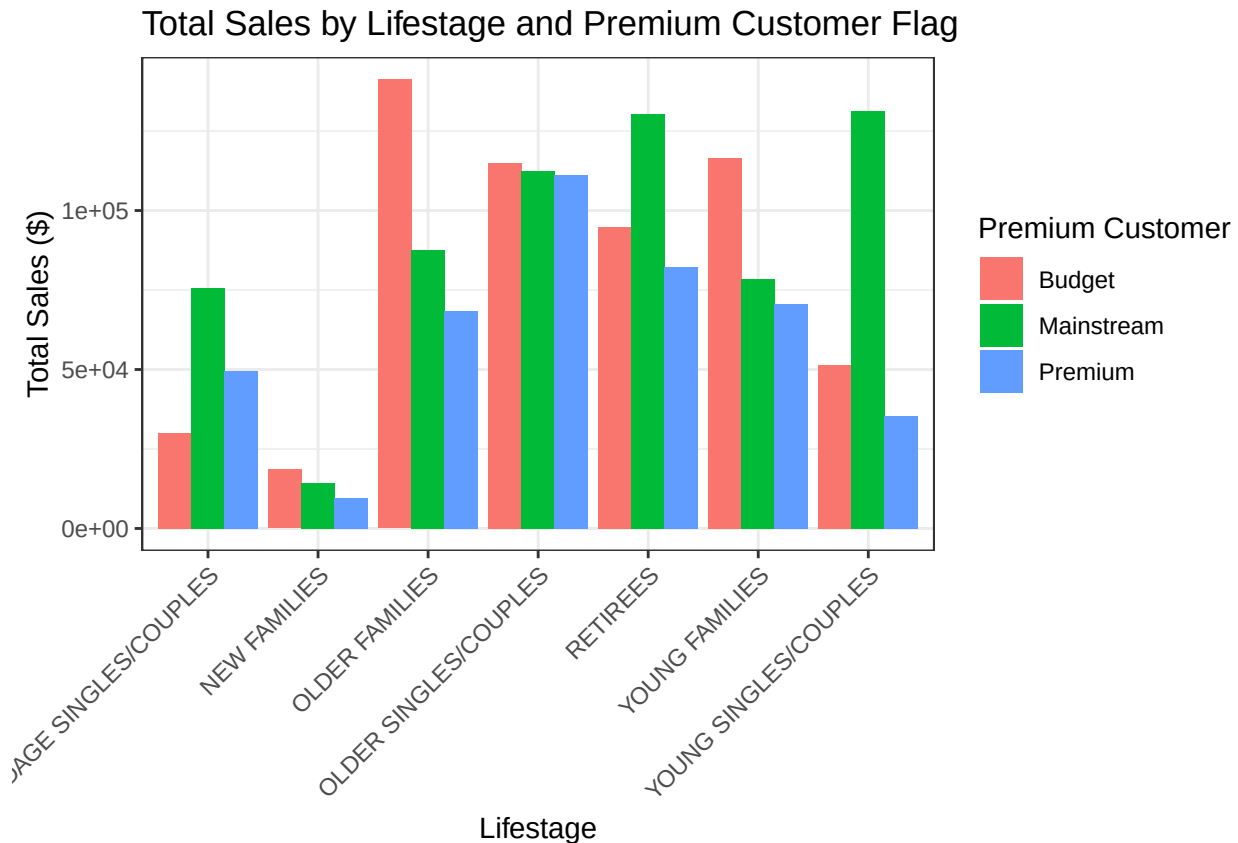
We have earlier seen in the previous few lines of codes that the mainstream buyers are our biggest client base, but that was a general summary data. Perhaps we would be interested in knowing how well they perform when compared with the budget and premium buyers according to their life stages.

```
sales_summary <- mergedData %>%
  group_by(lifestage, premium_customer) %>%
  summarise(total_sales = sum(tot_sales, na.rm = TRUE), .groups = "drop")
```

```

sales_summary %>%
  ggplot(aes(x = lifestage, y = total_sales, fill = premium_customer)) +
  geom_col(position = "dodge") +
  labs(title = "Total Sales by Lifestage and Premium Customer Flag",
       x = "Lifestage",
       y = "Total Sales ($)",
       fill = "Premium Customer") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



The striking variable in this chart is that the “New families” are under performing. For whatever reason, they are not buying nearly enough chips to balance out the scales. And besides the New families and the middle aged singles/couples, the other segments have fairly competitive statistics. Older families and young families are composed mainly of budget buyers, older singles/couples are equally budget, mainstream or premium buyers, retirees and young singles/couples are mostly mainstream buyers. *I will compare this finding with the number of customers that belong to each of these life stages and their spending habits.*

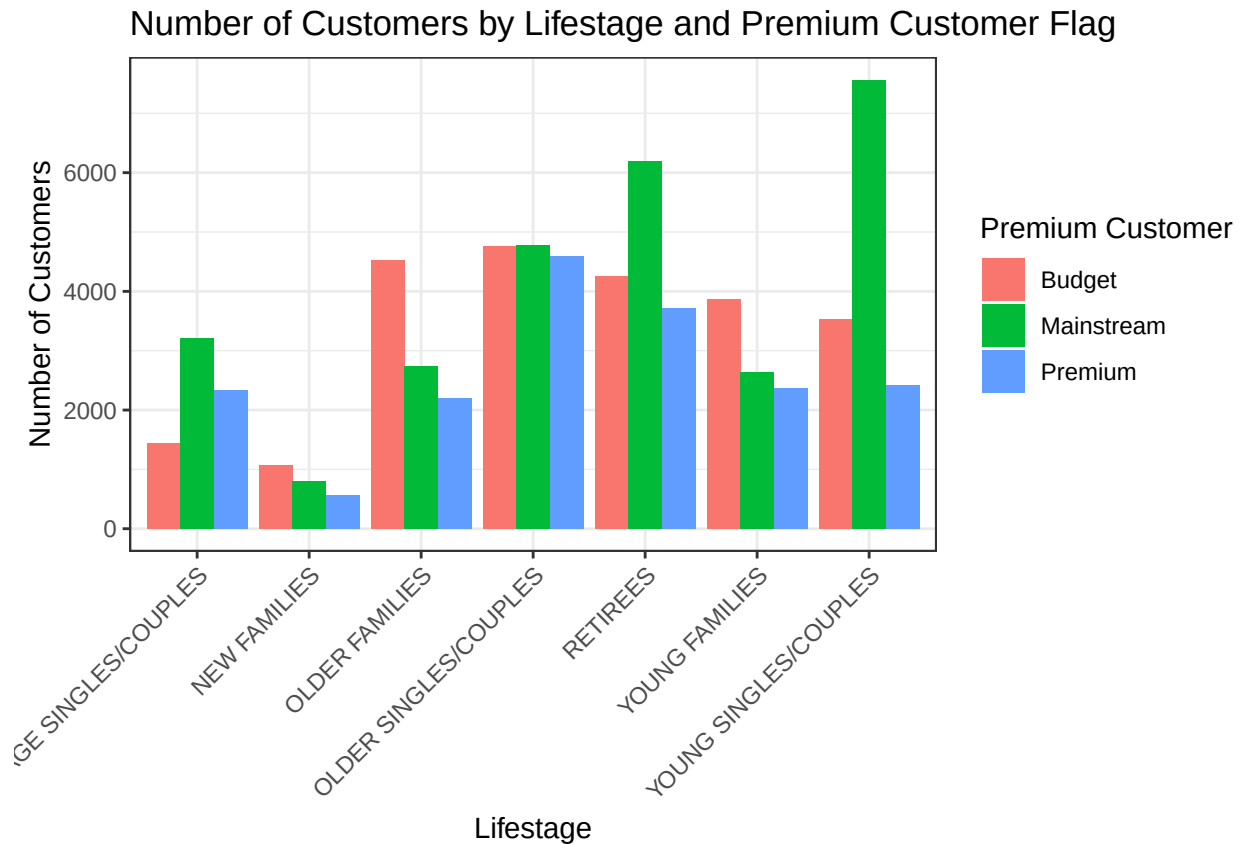
```

customer_summary <- mergedData %>%
  group_by(lifestage, premium_customer) %>%
  summarise(num_customers = n_distinct(lylty_card_nbr)) %>%
  ungroup()

customer_summary %>%
  ggplot(aes(x = lifestage, y = num_customers, fill = premium_customer)) +
  geom_col(position = "dodge") +
  labs(title = "Number of Customers by Lifestage and Premium Customer Flag",

```

```
x = "Lifestage",
y = "Number of Customers",
fill = "Premium Customer") +
theme_bw() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



This chart does not completely show a direct relationship between the number of customers and the total sales per transaction across the different customer life stages and spending category. While young singles/couples have the most number of mainstream buyers and maintain the highest revenue from the mainstream category, they are not the biggest source of revenue. That would be the older families who are budget buyers, however, they do not make up the proportionate amount of customers expected to be seen with the revenue they generate.

3.3.6 How much money is being generated over time and by whom?

Here, I wanted a snapshot of how many transactions Mainstream customers make, total revenue they generate, and their average spend, with a side by side comparison of premium and budget customers.

```
mergedData %>%
  group_by(premium_customer) %>%
  summarise(
    total_transactions = n(),
    total_revenue = sum(tot_sales, na.rm = TRUE),
    avg_spend = mean(tot_sales, na.rm = TRUE)
```



```
) %>%
arrange(desc(total_revenue))
```

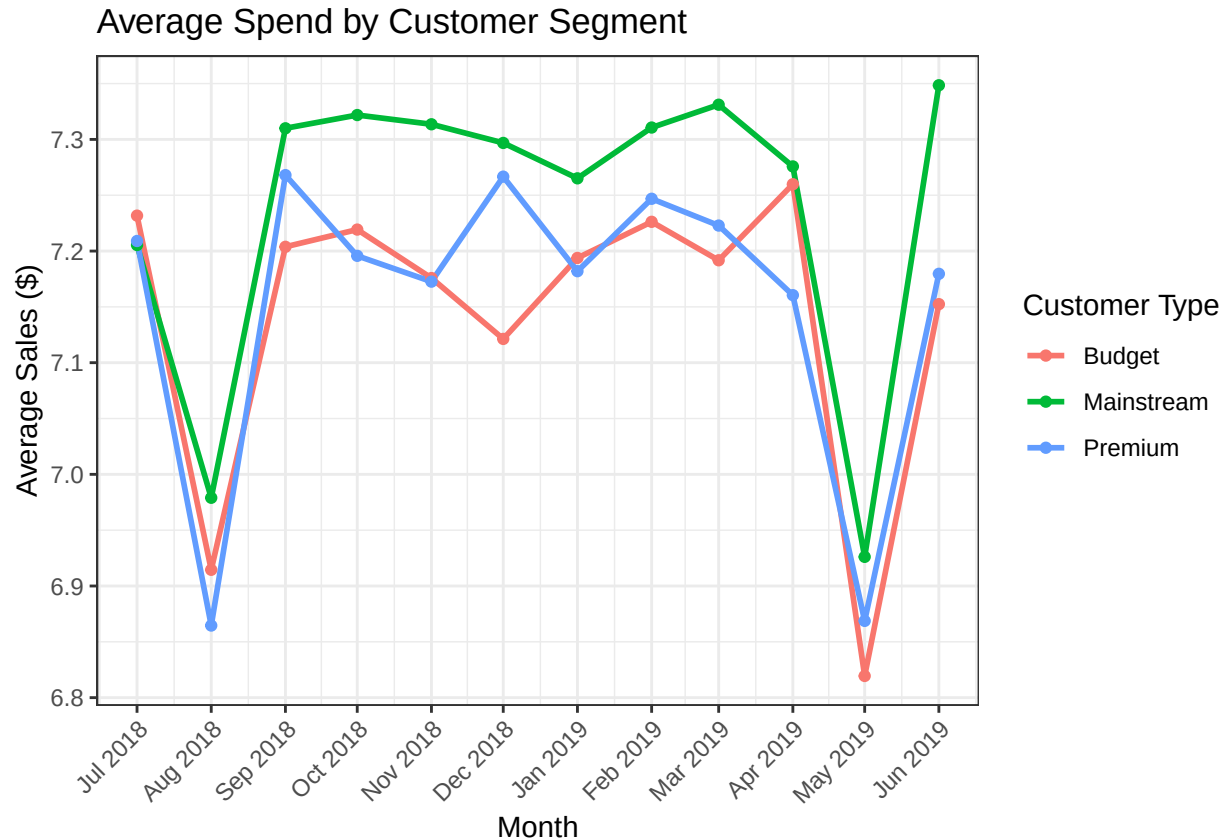
```
## # A tibble: 3 x 4
##   premium_customer total_transactions total_revenue avg_spend
##   <chr>              <int>          <dbl>    <dbl>
## 1 Mainstream         87026        629996.     7.24
## 2 Budget             79471        567517.     7.14
## 3 Premium           59685        426880.     7.15
```

As expected, our customers with the largest number of transactions (the mainstream buyers) are the ones generating most of the revenue. *Priority is to be placed on the mainstream customers and the type of chips they prefer to buy.*

3.3.6b Visual representation of consumer transaction trend

```
CustomerSpending <- mergedData %>%
  mutate(month = floor_date(date, "month")) %>%
  group_by(month, premium_customer) %>%
  summarise(avg_sales = mean(tot_sales, na.rm = TRUE))
```

```
ggplot(CustomerSpending, aes(x = month, y = avg_sales, color = premium_customer)) +
  geom_line(size = 1) +
  geom_point() +
  labs(title = "Average Spend by Customer Segment",
       x = "Month",
       y = "Average Sales ($)",
       color = "Customer Type") +
  scale_x_date(date_breaks = "1 month", date_labels = "%b %Y") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



This chart shows the average spend per transaction across the three customer segments over the 12 month period. There is a majorly consistent price pattern throughout the year, and Mainstream customers remained the highest spenders on average, with up to \$7.35 per transaction. Premium and Budget customers are following behind, but all three of them spent considerably less in the months of August 2018 and May 2019. This generalized dip followed by a sharp recovery suggests an external influence like promotional sales, or seasonal variations. *Mainstream customers are the most valuable clients here, because they don't just frequently purchase products, but they also spend more than their premium and budget counterparts when they do.*

3.3.7 Do the pack sizes influence the sales?

We might want to know if the different size of chips being bought is influencing the total revenue or average spending of customers.

```
mergedData %>%
  filter(!is.na(pack_size)) %>%
  group_by(pack_size) %>%
  summarise(
    transactions = n(),
    total_sales = sum(tot_sales, na.rm = TRUE),
    avg_sale_value = mean(tot_sales, na.rm = TRUE)
  ) %>%
  arrange(desc(total_sales))
```

```
## # A tibble: 19 x 4
```

	pack_size	transactions	total_sales	avg_sale_value
	<chr>	<int>	<dbl>	<dbl>
## 1	175g	61892	465082.	7.51
## 2	150g	38683	282099.	7.29
## 3	134g	25102	177656.	7.08
## 4	110g	22387	162765.	7.27
## 5	165g	15297	101361.	6.63
## 6	170g	13606	92927	6.83
## 7	380g	6416	75420.	11.8
## 8	330g	6339	69107.	10.9
## 9	270g	3170	27853	8.79
## 10	135g	3257	26090.	8.01
## 11	300g	5842	22546.	3.86
## 12	210g	3167	21701.	6.85
## 13	160g	2970	10648.	3.59
## 14	200g	2964	10617.	3.58
## 15	90g	3008	9676.	3.22
## 16	190g	1479	9243.	6.25
## 17	180g	1468	8568.	5.84
## 18	70g	1507	6852	4.55
## 19	220g	1564	6831	4.37

The most bought pack size is the 175g chip, which has been bought a remarkable 23,209 times more than the next leading pack size. It also dominates the total revenue brought in, even though its average sales value is not the highest. The least bought is the 180g, which is surprising because the sizes are quite close to each other, and its total revenue is higher than the last two sizes with the least amount of revenue.

To visualize this, we first we have to strip the letter “g” off of all the pack sizes, to allow our geom code to be able to run. I will essentially be creating a new column where the pack sizes have a numeric class rather than a character one.

```
mergedData <- mergedData %>%
  mutate(pack_size_num = as.numeric(gsub("g", "", pack_size)))
```

To confirm that this actually worked:

```
class(mergedData$pack_size_num)
```

```
## [1] "numeric"
```

```
head(mergedData$pack_size_num)
```

```
## [1] 175 170 175 150 330 210
```

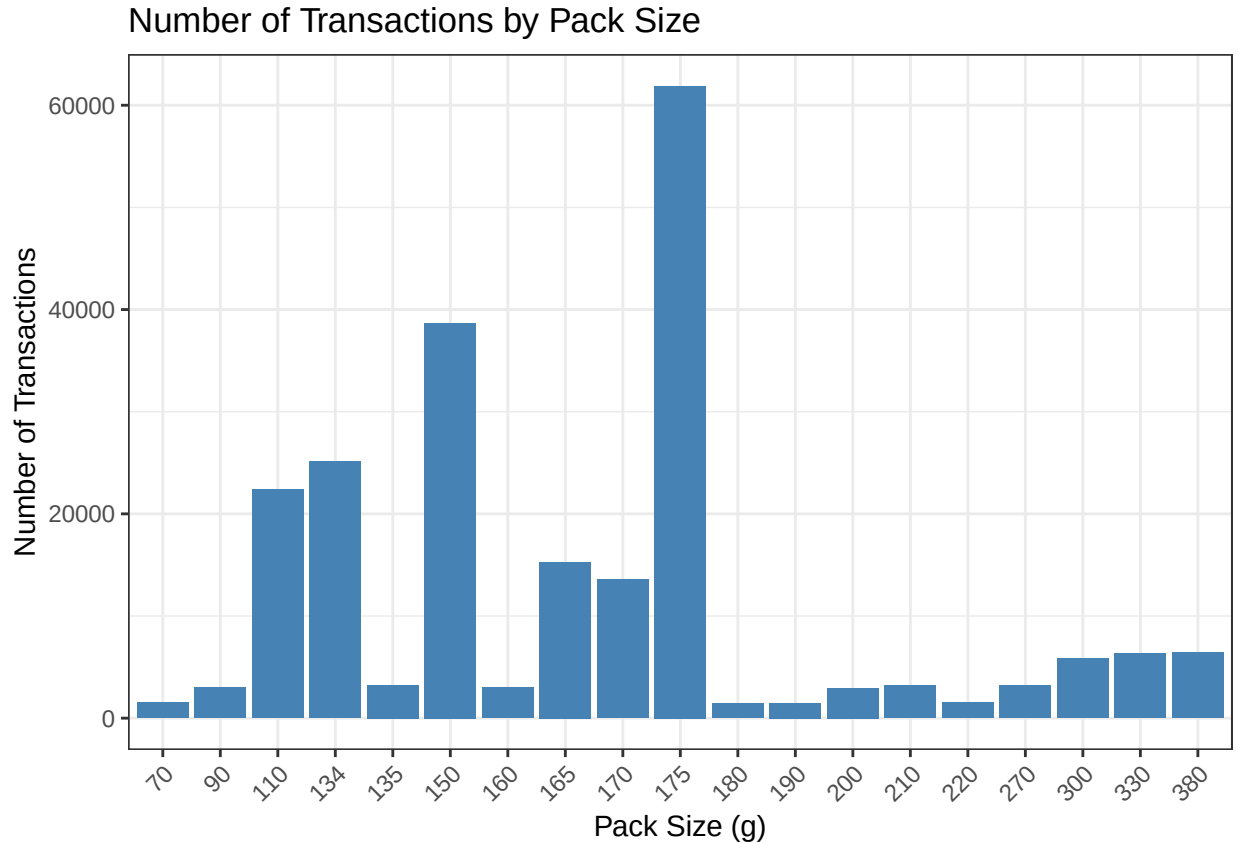
Now, to plot our data

```
mergedData %>%
  filter(!is.na(pack_size_num)) %>%
  count(pack_size_num) %>%
  ggplot(aes(x = factor(pack_size_num), y = n)) +
  geom_col(fill = "steelblue") +
  labs(title = "Number of Transactions by Pack Size",
```

```

x = "Pack Size (g)",
y = "Number of Transactions") +
theme_bw() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



175g dwarfs everything else on the chart at over 60,000 transactions. 150g is a strong second at nearly 40,000. Both 175g and 150g likely account for close to half of the total transaction volume, given how far ahead they are of the rest. 110g and 134g have quite the meaningful volume of transactions, outperforming 165g and 170g, which are the last two that ranked above 10,000 transactions. The rest of the sizes appear to be the less popular pack sizes, although the larger sizes (300g, 330g, 380g) show a small uptick compared to the mid-range ones, suggesting there's a modest demand for bulk/family-size packs, distinct from the standard-size cluster. *I would recommend that 175g and 150g sizes are prioritized above all the others, seeing as they do all the heavy lifting. Treat 110g, 134g, 165g and 170g as steady sellers and maintain consistency of the volume of their stock. If the rest of the sizes perform poorly because they are products that are being phased out, it might be worth removing them from the shelf or replacing them with a newer product. However, I do think there might be an under explored opportunity in the family size niches that is not being capitalized on.*

3.3.8 What about the number of chips being bought?

Are customers buying one pack at a time, or multiple per transaction? To find out, I first have to determine which transactions were for single packets, and which were for multiple packets.

```
mergedData <- mergedData %>%
  mutate(pack_type = ifelse(prod_qty == 1, "Single Pack", "Multi Pack"))
```

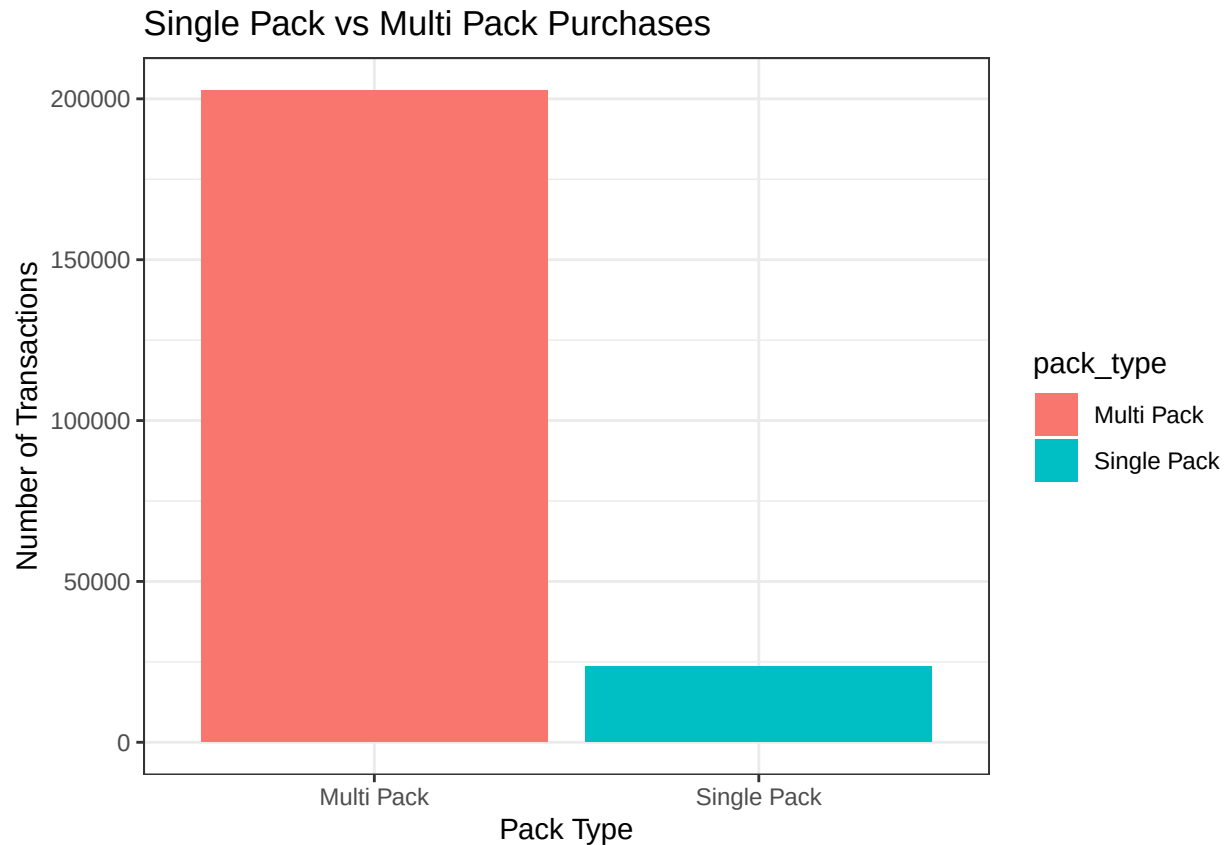
I created a new column using the product quantity column, so that where one packet of chip was bought, it becomes a single pack transaction, and 2 or more packets of chips bought gets labelled a multipack transaction.

```
mergedData %>%
  group_by(pack_type) %>%
  summarise(
    total_transactions = n(),
    unique_customers = n_distinct(lylty_card_nbr),
    avg_spend = mean(tot_sales, na.rm = TRUE)
  )
```

```
## # A tibble: 2 x 4
##   pack_type    total_transactions unique_customers avg_spend
##   <chr>          <int>          <int>      <dbl>
## 1 Multi Pack      202639          60335       7.59
## 2 Single Pack     23543          19012       3.66
```

Here, we see that the total number of transactions where more than one packet of chips was bought far exceeds the number of transactions for a single chip. And the average sales is also bigger, and it would seem that a good majority of our customers prefer to buy more than one pack of chips.

```
mergedData %>%
  count(pack_type) %>%
  ggplot(aes(x = pack_type, y = n, fill = pack_type)) +
  geom_bar(stat = "identity") +
  labs(title = "Single Pack vs Multi Pack Purchases",
       x = "Pack Type",
       y = "Number of Transactions") +
  theme_bw()
```

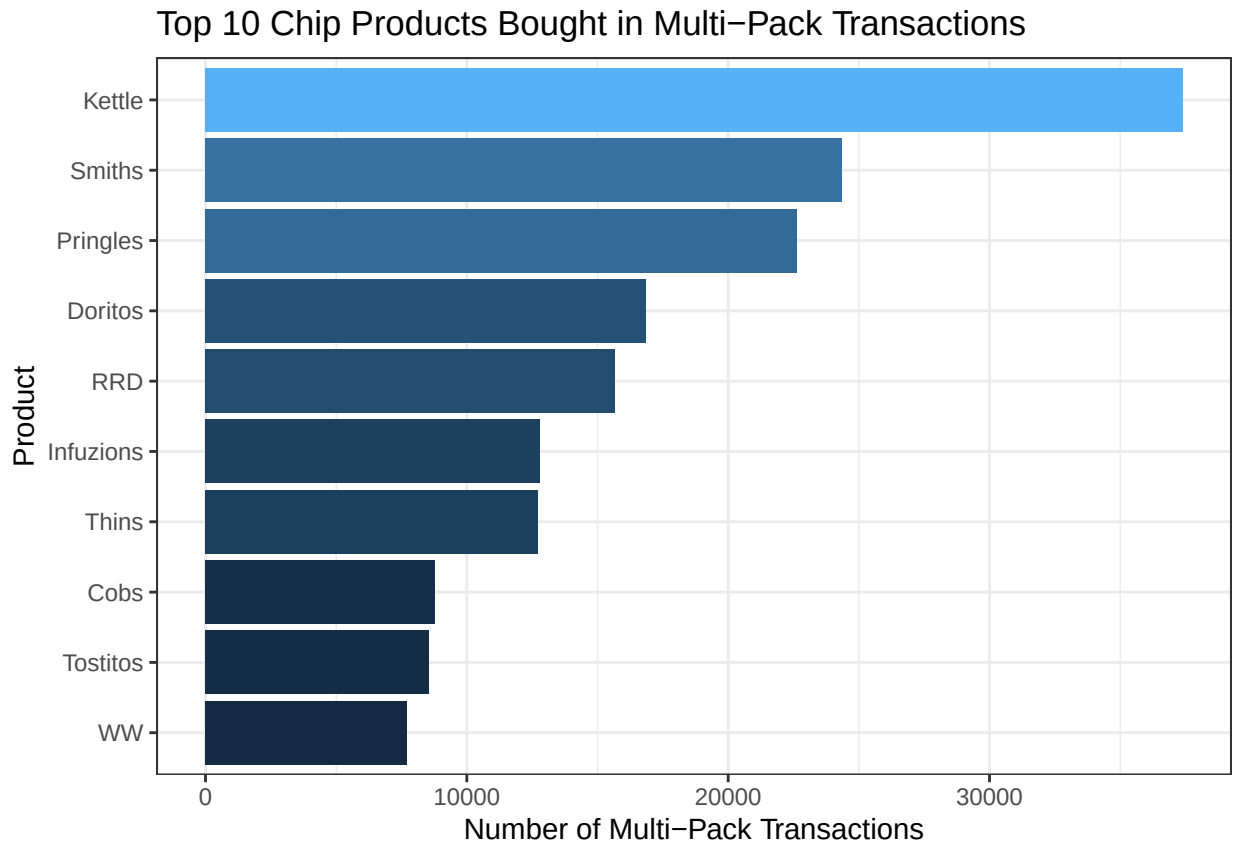


This chart even more clearly shows a staggering difference in the two segments, with over 200,000 customers preferring to buy multiple packets instead of just one at a time. *Business is good, the stock has to remain available because customers want more. I would recommend paying close attention to the type of chips that are likely to be bought as multiple packets in a single transaction and increase their available stock.*

3.3.9 What chips Brands are being bought in multiples per transaction?

Instead of just recommending that you look into what chips customers are buying more of in a single transaction, I should show you.

```
mergedData %>%
  filter(prod_qty > 1) %>%
  count(Brand, sort = TRUE) %>%
  top_n(10, n) %>%
  ggplot(aes(x = reorder(Brand, n), y = n, fill = n)) +
  geom_col() +
  coord_flip() +
  labs(title = "Top 10 Chip Products Bought in Multi-Pack Transactions",
       x = "Product",
       y = "Number of Multi-Pack Transactions") +
  theme_bw() +
  theme(legend.position = "none")
```

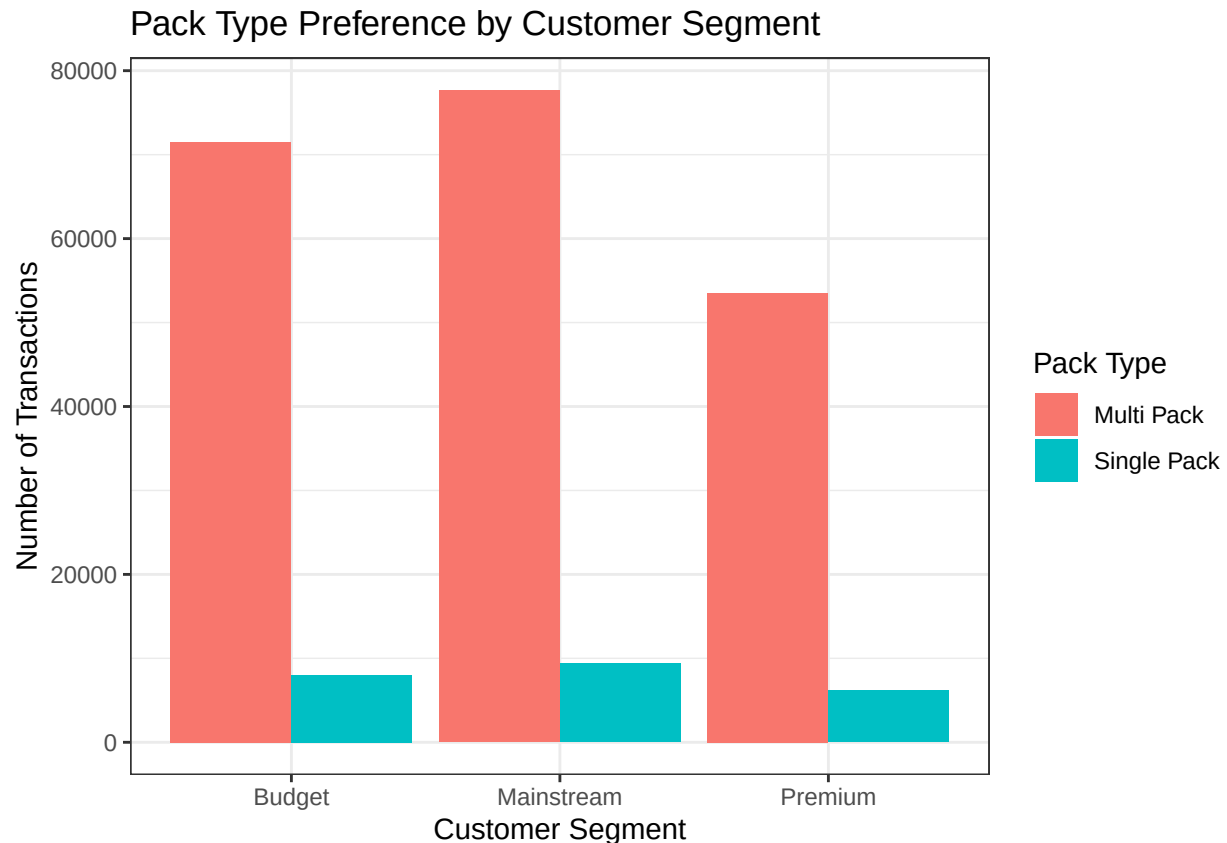


As suspected, the same products that ranked our most popular in the last few analysis, also ranked the most number of multi-packs transactions. *These should remain the focus point of our business. These brands should be restocked as quickly as they may have a tendency to sell out just as quickly, and we want to make sure that the customers keep coming.*

3.3.10 Who prefers buying multiple packs at a time?

What kind of customers prefer to by multiple packs in a single transaction? Let us find out.

```
mergedData %>%
  group_by(premium_customer, pack_type) %>%
  summarise(total_transactions = n()) %>%
  ggplot(aes(x = premium_customer, y = total_transactions, fill = pack_type)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Pack Type Preference by Customer Segment",
       x = "Customer Segment",
       y = "Number of Transactions",
       fill = "Pack Type") +
  theme_bw()
```



Even across the three different customer segments, the verdict is the same. People just prefer to buy their chips in multiples at a time. *Mainstream buyers are our biggest customers still, so we might want to focus on them, but still keep the premium buyers in mind too, because they also have a bigger number of transactions when compared to budget buyers.*

4. Recommendations

Based on my analysis and findings from the data provided, these are my recommendations: *Seasonal promotions should be leveraged from late November to capitalize on the Christmas sales surge. Future marketing campaigns should be crafted with the mainstream buyers as the major customer base, and curate a strategy to enhance premium sales as well. Since customers prefer to buy multiple chips per transaction, consider discount offers catered to this preference in order to maximize sales.*

```
write_csv(mergedData, here("Data", "mergedData.csv"))
```